

Datové struktury a algoritmy

Přednáška 7, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

Dnešní témata

- Prezentace:
 - **Case sort, B stromy**; B+ stromy; sekvenční vyhledávání; vyhledávání binárním půlením; binární vyhledávací stromy, obecné vlastnosti
- Bonifikovaný kvíz: Halda a prioritní fronta
- Algoritmy řazení (dokončení)
 - Krabičkové a indexové řazení (kvazilineární metody)
 - B-stromy
 - Vnější řazení (disková data)
- Problematika vyhledávání, základní principy
- Úvod do binárních vyhledávacích stromů

B-stromy

- Vymyslel R. Bayer v roce 1970
 - K čemu jsou dobré: pro ukládání vyhledávacích stromů do externí paměti (do diskových souborů)
 - Princip: zvětšíme uzly a uložíme do nich více klíčů a ukazatelů na potomky
 - Každý uzel bude obsahovat r až $2r$ klíčů (položek) a $r+1$ až $2r+1$ ukazatelů na potomky
 - Složitost operace vyhledávání je v takovém stromu v nejhorším případě $\log_r(N)$, kde N je počet klíčů ve stromu
-
- Vazba mezi B stromy a vyhledávacími algoritmy:
 - 2-3-4 B-strom (uzel obsahuje 1 – 3 hodnoty, tj. může mít 2 – 4 ukazatele) je alternativní reprezentací totožného **váhově vyváženého** RB stromu (= varianta BVS)
 - B-strom strom řádu $r = 1$ se nazývá 1-2 strom a je to **výškově vyvážený** BVS.

B-stromy

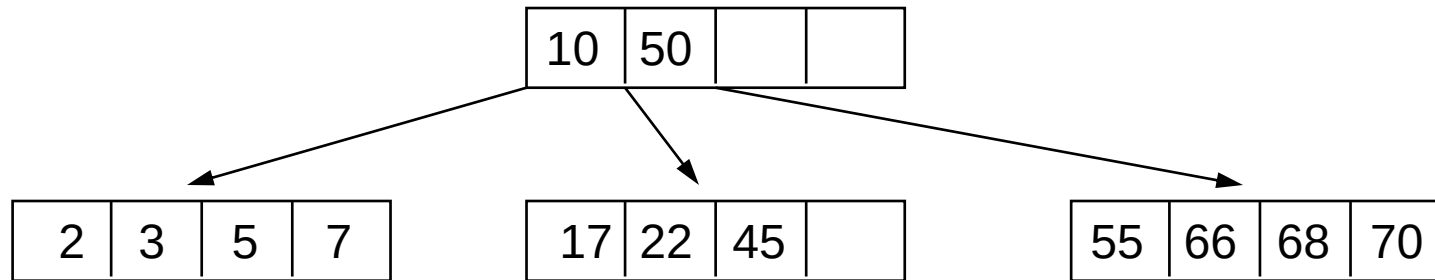
- Definice:

B-strom řádu r je $(2r+1)$ -ární vyhledávací strom, který splňuje následující podmínky:

- každý uzel obsahuje nanejvýš $2r$ klíčů (položek)
 - každý uzel, s výjimkou kořene, obsahuje alespoň r klíčů
 - každý uzel je buď listem (tj. nemá žádné potomky), nebo má $m+1$ potomků, kde m je počet klíčů v uzlu a platí $r \leq m \leq 2r$
 - všechny listy mají stejnou hloubku (jsou na stejné úrovni)
- Uzel B-stromu se někdy nazývá *stránka* (page)
 - souvisí to s tím, že stránka (blok) je jednotkou přenosu mezi diskovým souborem a vnitřní pamětí

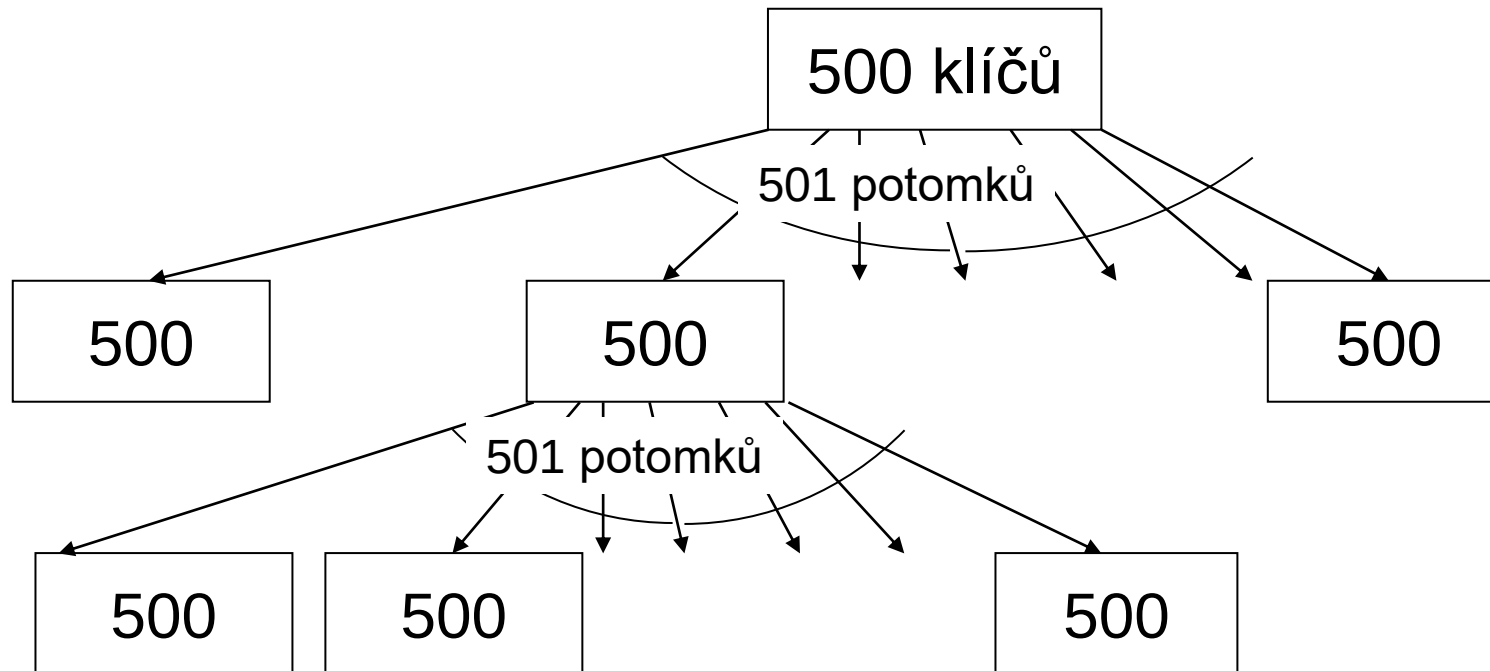
B-stromy

- Obsahuje-li uzel B-stromu m klíčů $k_1, k_2, \dots, k_m$, pak platí
$$k_1 < k_2 < \dots < k_m$$
- Obsahuje-li uzel B-stromu, který není listem, m klíčů, pak pro každé i , $1 \leq i \leq m$ platí
 - uzly v potomku p_{i-1} mají klíče menší než k_i
 - uzly v potomku p_i mají klíče větší než k_i
- Příklad B-stromu řádu 2:



B-stromy

- B strom řádu 500 hloubky 2, jehož uzly obsahují 500 klíčů (min. zaplnění)
 - strom obsahuje 501 podstromů kořenu (250 500 klíčů v hloubce 1)
 - každý podstrom obsahuje 501 podstromů (12 550 050 klíčů v hloubce 2)
 - celkem 12 801 150 klíčů



Hledání v B-stromu

- Ve stromu s kořenem r hledáme uzel s klíčem x
- Postup:
 - necht' uzel u obsahuje m klíčů
 - jestliže uzel u obsahuje klíč x , hledání je úspěšné
 - jinak:
 - je-li $x < k_1$, hledání pokračuje v potomku p_0
 - je-li $x > k_m$, hledání pokračuje v potomku p_m
 - jestliže platí $k_i < x < k_{i+1}$, pro $1 \leq i < m$, pak hledání pokračuje v následníku p_i

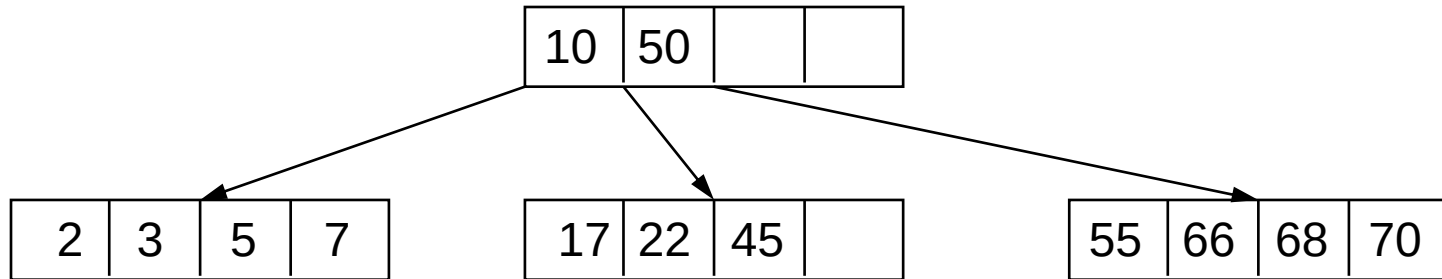
Vkládání klíče do B-stromu

- Zjistíme, zda se klíč ve stromu vyskytuje
- Pokud ano, nic se neděje (případně chyba)
- Pokud ne, je třeba klíč vložit do vhodného uzlu
- Mohou nastat tři případy:
 - vkládá se do **listu, který není plný** $\Rightarrow$ **zařad' podle klíče**
 - vkládá se do **listu, který je plný**; pak musí nastat **rozštěpení (split) listu**, a jeden klíč (median, tj. “klíč uprostřed”) musí být vložen do rodiče
 - vkládá se do listu, který je plný; pak musí nastat rozštěpení listu, a jeden klíč musí být vložen do **rodiče, který je rovněž plný, pak se i on rozštěpí**. Rozštěpí-li se kořen stromu, výška stromu se zvýší o 1.
- Poznámka:

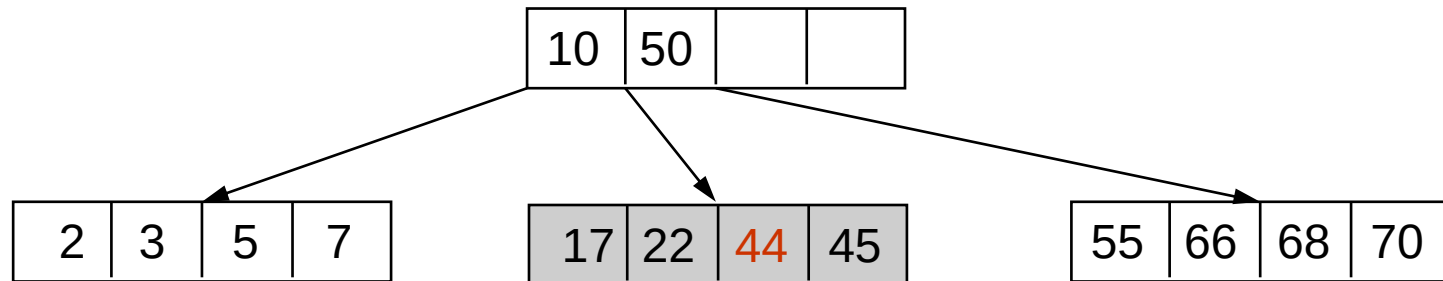
Při štěpení uzlu stromu řádu r , který obsahuje klíče $k_1, k_2, \dots, k_m$ ($m=2r$), se do uspořádané posloupnosti klíčů vloží nový klíč, takže vznikne posloupnost délky $2r+1$. Prvních r klíčů bude součástí prvního nového uzlu, posledních r klíčů bude součástí druhého nového uzlu, a prostřední klíč bude zaslán k vložení rodiči uzlu, který se štěpí.

Vkládání klíče do B-stromu

- Vložení bez štěpení uzlu

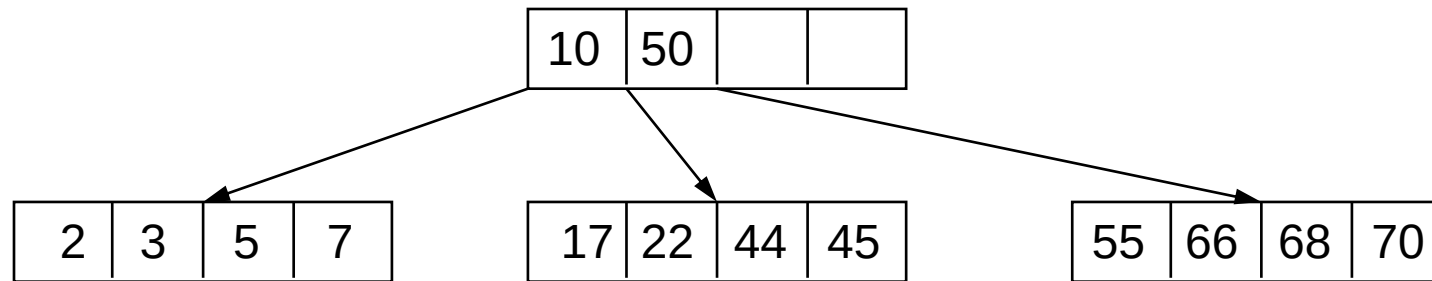


ins (44)

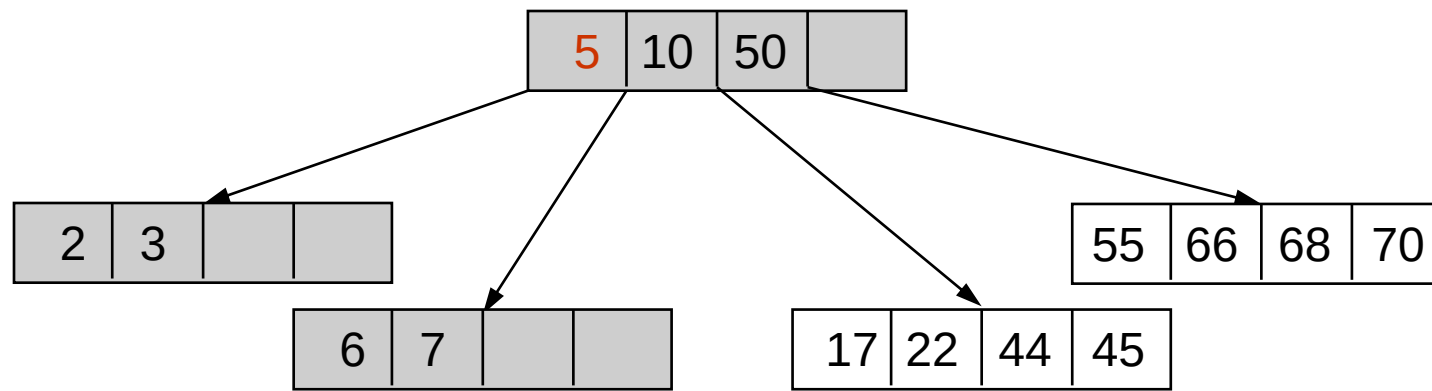


Vkládání klíče do B-stromu

- Vložení se štěpením uzlu

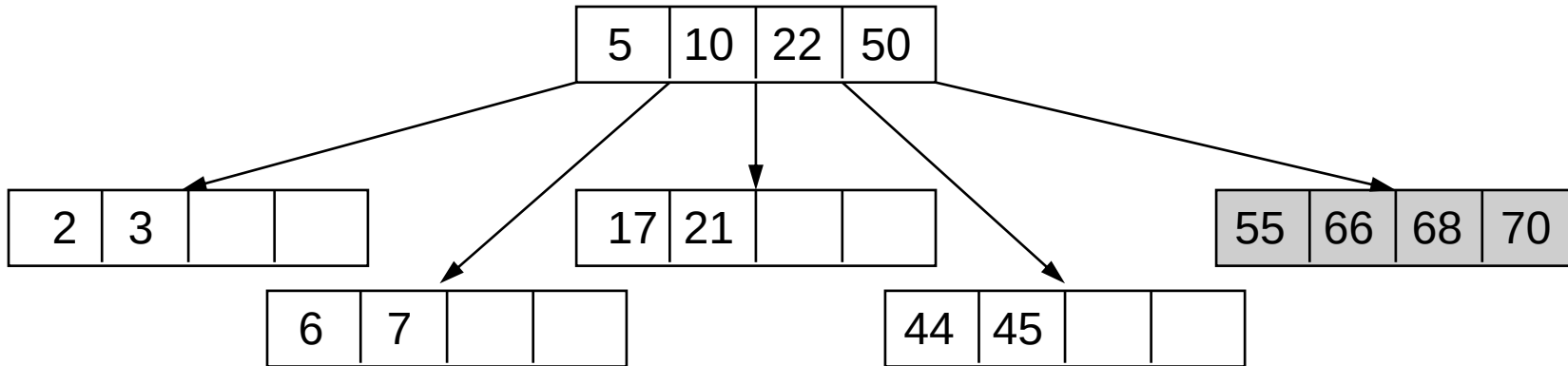


ins (6) (štěpí se uzel pro klíče 2 3 **5** 6 7)

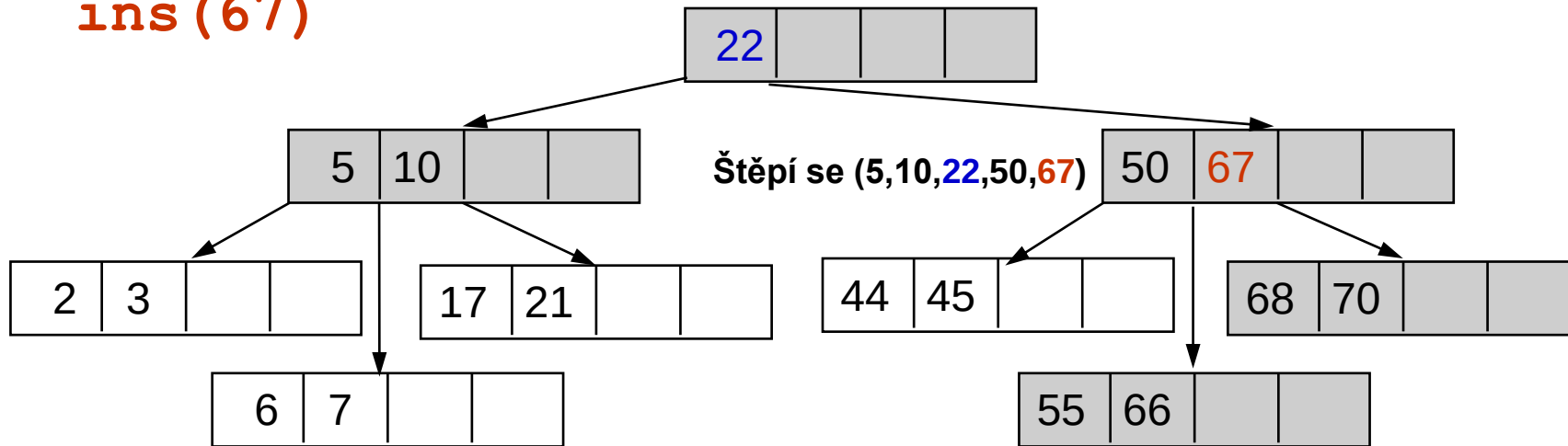


Vkládání klíče do B-stromu

- Vložení se štěpením uzlu a zvětšením výšky stromu



ins (67)



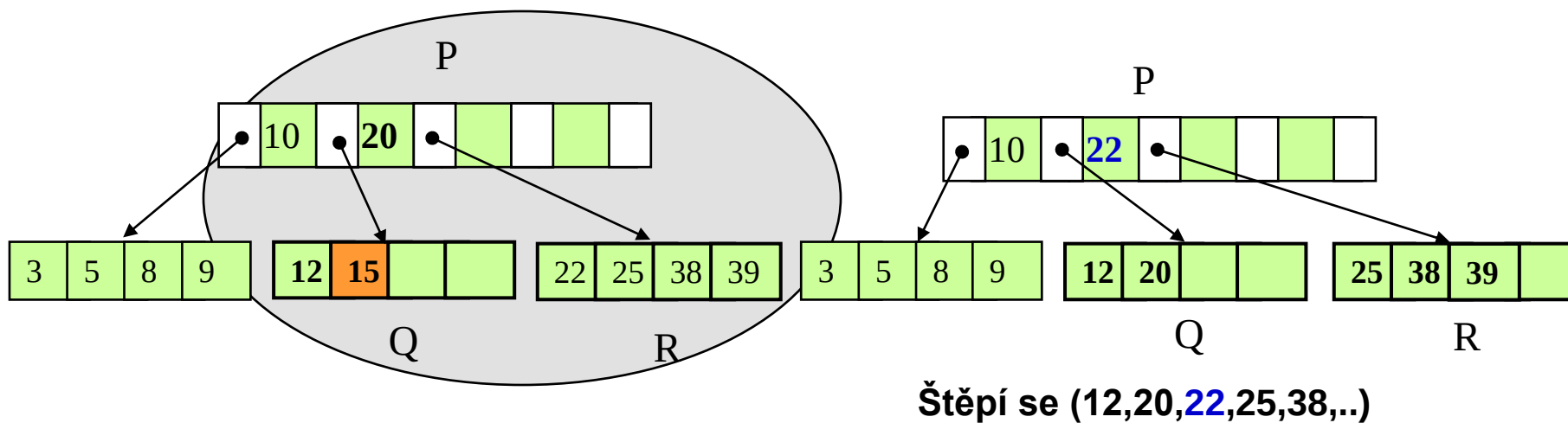
Rušení prvku v B-stromu

- **Rušení prvku v B- stromu**

1. Nalezení rušeného prvku $\Rightarrow$ uzel P
2. Jeli rušený prvek v listu
pak vyjmi prvek z listu
jinak nahraď rušený prvek nejbližším nižším prvkem
v nejpravějším listu Q levé větve rušeného prvku
(analogicky náhrada nejbližším vyšším prvkem)
3. Měl-li Q před rušením m prvků
pak VYVAŽUJ

Rušení prvku v B-stromu

- **Vyvažování po rušení prvku v B-stromu**



- **Složitost**

$$Q(n) = I(n) = D(n) = O(\log_r n)$$

B⁺-stromy

- **B⁺- strom**

Zřetěžený seznam seřazených prvků + vyhledávací B-strom

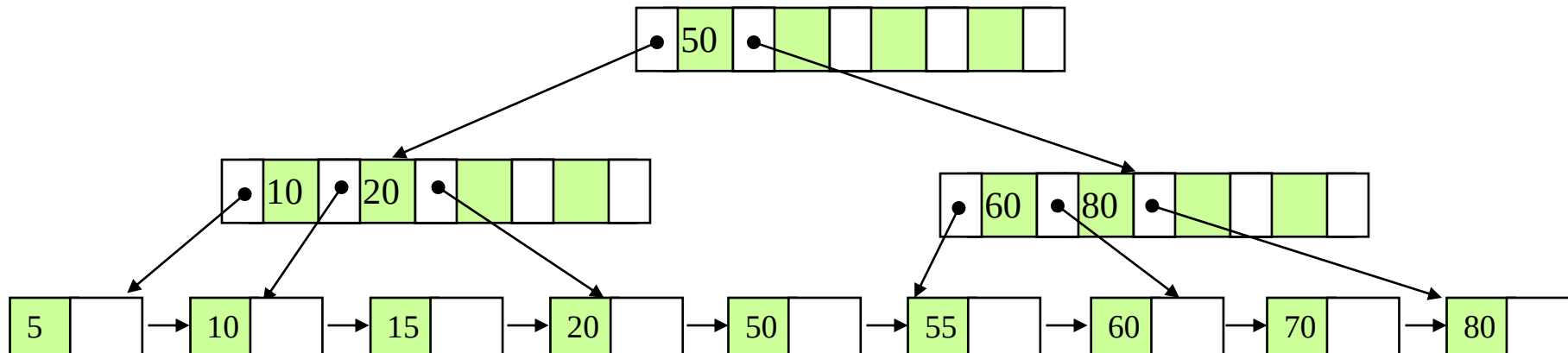
B-strom obsahuje pouze klíče, nikoliv hodnoty prvků

- Motivace:

Vyšší řád r vyhledávacího B-stromu při dané délce stránky

⇒ nižší výška

⇒ méně přístupů na disk



Štěpení B-stromů

- **B-strom** stavíme shora a v paměti/uzlu vzestupně nebo sestupně řadíme klíče, odkazující na disková data
- **B+ strom** stavíme zdola, tj. od dat
 - Tato struktura spojuje výhodu stromového indexování se sekvenčním řazením datových uzlů, usnadňuje vyhledávání podle rozsahu a vykazuje vyšší průměrnou míru zaplnění paměťových stránek ve srovnání s klasickým B-stromem. V praxi se využívá její ještě více paměťově optimalizovaná varianta **B***

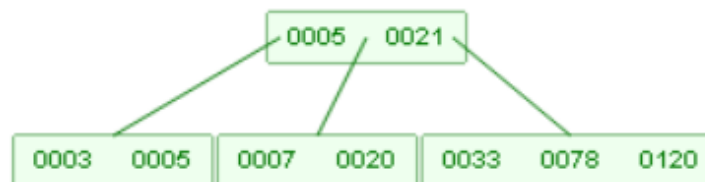
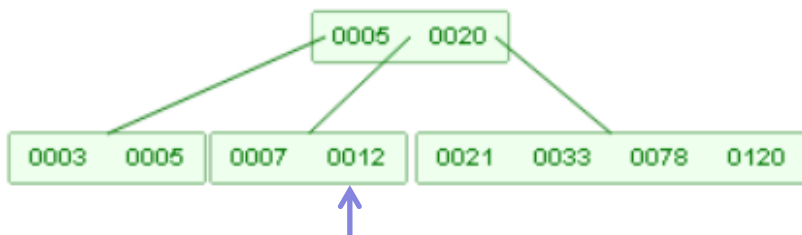
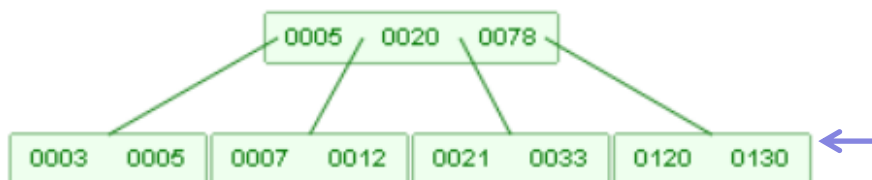
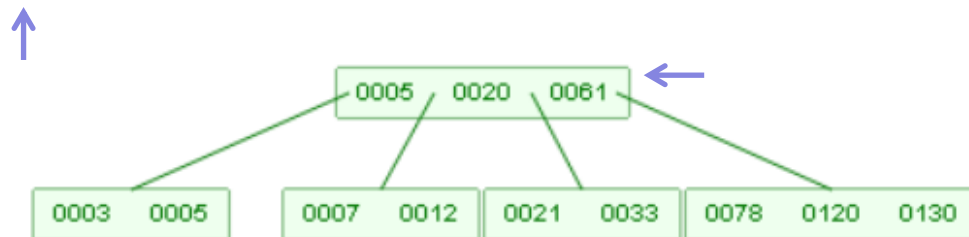
Vkládání do B-stromu

- Je-li místo v uzlu, pouze přidáme nový klíč
- Je-li místo v rodičovském uzlu, rozštěpíme naddimenzovaný uzel horizontálně a medián celé množiny klíčů uzlu přesuneme o úroveň výš
- Není-li o úroveň výše místo, přidáme další úroveň

Odebírání

- Zůstane-li v uzlu po odebrání více než polovina (nebo celá část) ostatními klíči obsazených pozic, konkrétní klíč pouze odebereme
- Zůstane-li jich méně než polovina, vyvažujeme strom horizontálně, a je-li to nutné i vertikálně (strom se napřed zužuje a pak snižuje)

B-strom řádu 5, příklady rušení klíčů



del(2): žádná změna struktury - klíč odebíráme z dostatečně zaplněného listového uzlu

del(61): 61 nahradíme 33 (nejbližší nižší), musíme dovážit uzel s 21, tj. novou strukturu (21,33,78,120,130), což vede na přesun 78 o úroveň výše (medián) a vytvoření uzlů (21,33) a (120,130) ze zbylých klíčů

del(130): kvůli nedostatku klíčů v uzlu se zbylou 120 dovažujeme novou strukturu (21,33,78,120), která se vejde do jediného uzlu

del(12): dovažujeme (7,20,21,...), což vede na přesun 21 nahoru a související aktualizaci obou listů.

B-stromu řádu 5, hands-on příklad

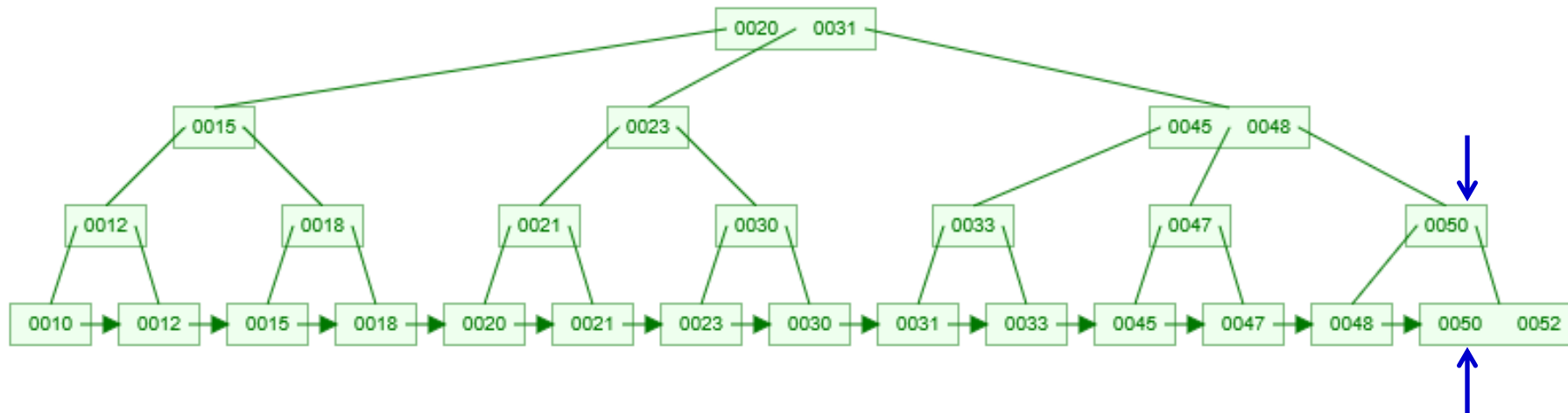
- Klíče: 1, 12, 8, 2, 25, 5, 14, 28, 17, 7, 52, 16, 48, 68, 3, 26, 29, 53, 55, 45



Paměťové klíče odkazují na zbytek jim odpovídajících záznamů, uložený na disku.

B+ stromu řádu 3, hands-on příklad

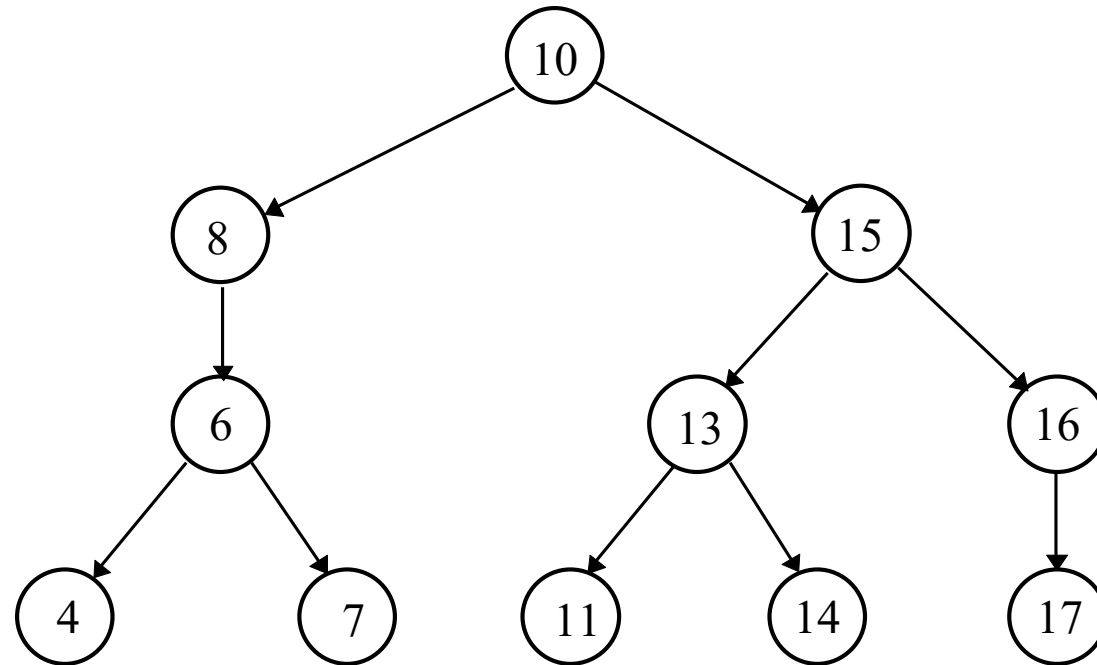
- Klíče: 10, 12, 15, 18, 20, 21, 23, 30, 31, 33, 45, 47, 48, 50, 52



Vnitřní uzly stromu reprezentují klíče, ovšem v jeho listech jsou navíc sekvenčně uspořádané odkazy na data (viz např. klíč 50 a datový blok 50).

1-2 strom

- Zvláštním případem B-stromu je B-strom strom řádu $r = 1$.
Nazývá se **1-2 strom** a je to výškově vyvážený BVS.



1-2 strom

Pro 1-2 strom platí:

1. všechny listy mají stejnou vzdálenost od kořene,
2. uzel, který má jednoho syna, má “bratra” se dvěma syny (odtud 1-2 strom).

Z druhé podmínky vyplývá:

- kořen 1-2 stromu je buď listem, nebo má dva syny,
- jediný syn **s** uzlu **u** musí mít dva syny (jinak by vnuk nemohl mít bratra se dvěma syny).

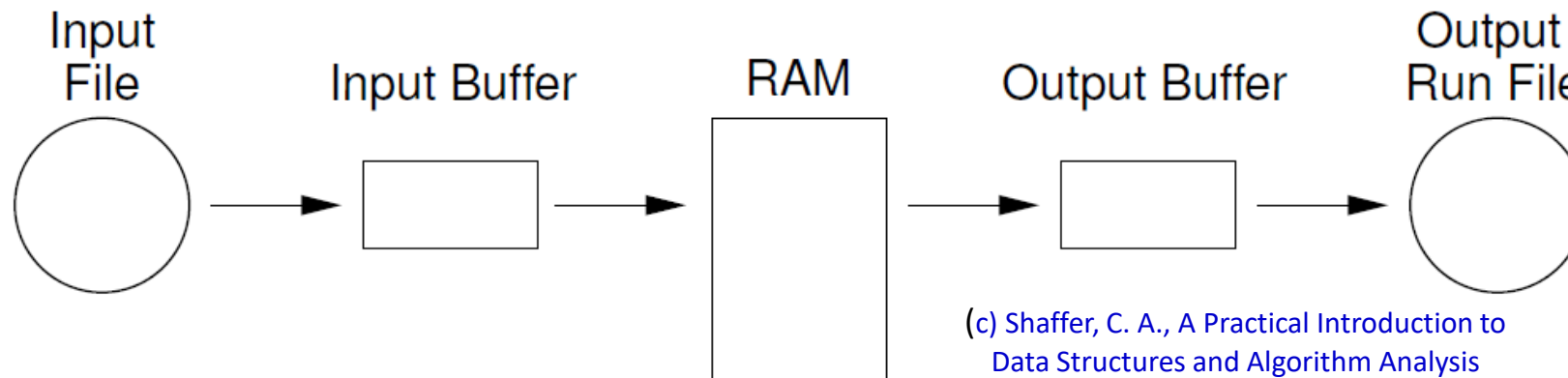
1-2 stromy jsou vhodné pro implementaci vyhledávacího problému v operační paměti

Externí řazení, základní úvahy

- Přístupová doba do paměti: **ns**
- Přístupová doba na pevný disk: **ms**
 - Logický (programátorský) a fyzický (HW) disk
 - FAT vs. i-node
 - Rozdílná HW architektura HD a CD-ROM
- Rozdíl několika řádů -> **minimalizace přístupů na disk**
 - Použití algoritmů vnitřního řazení na externí data
 - Přímě v paměti – nereálné
 - Využívají cache a buffery, disk pouze výjimečně
- Řešení:
 - Optimalizace souborového systému
 - Komprimace externích (diskových) dat
 - Využívání vyrovnávací paměti (data z blízkých sektorů)
 - Časová preference často adresovaných dat

Blokové externí řadící algoritmy

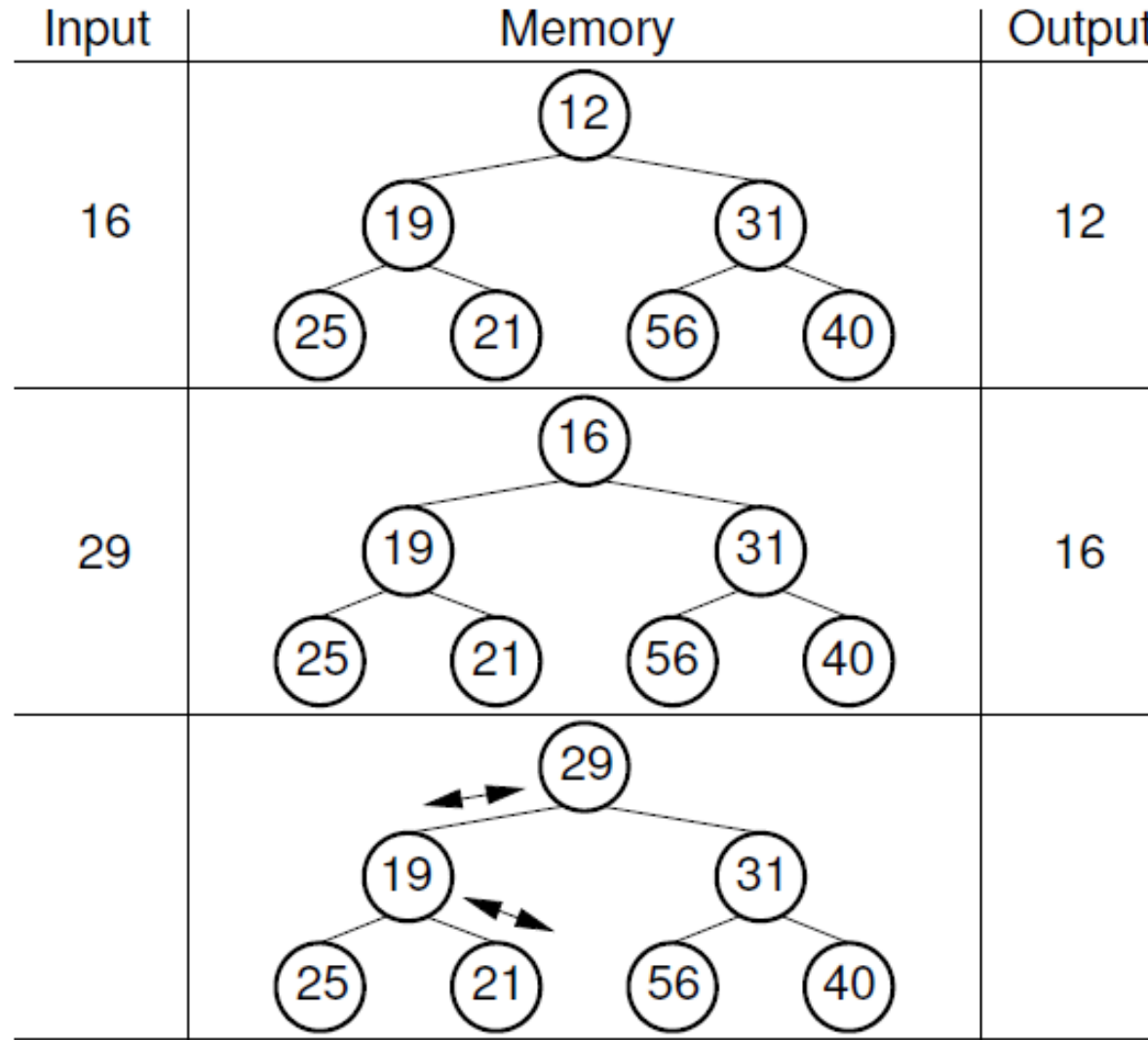
- Soubor o velikosti N je rozdělen do k bloků podle velikosti dostupné paměti m , kde $k = (N/m)+1$
- V prvním průchodu jsou jednotlivé bloky seřazeny v paměti (např. algoritmem HeapSort nebo QuickSort) a překopírovány na disk
- V dalších průchodech je na seřazené bloky aplikován **vícecestný Merge Sort**, slučující výstupy z $k-1$ dříve seřazených bloků
 - Počet průchodů: $PP = 1 + \log_{k-1}(N/m)$
 - I/O časová náročnost: $2N * PP$



- Pracuje sekvenčně, nepředpokládá znalost délky vstupních dat
- Využívá MIN-Heap v paměti RAM
- S novým vstupem se vrchol haldy (MIN) kopíruje na výstup
- Vstupy menší než MIN se v aktuálním průchodu ignorují
- Je-li nový vstup větší, zařadí ho do haldy a při dalším vhodném vstupu ho kopíruje na výstup
- Vytváří tak řadu dílčích setříděných seznamů, které nakonec sloučí (merge) do jediného

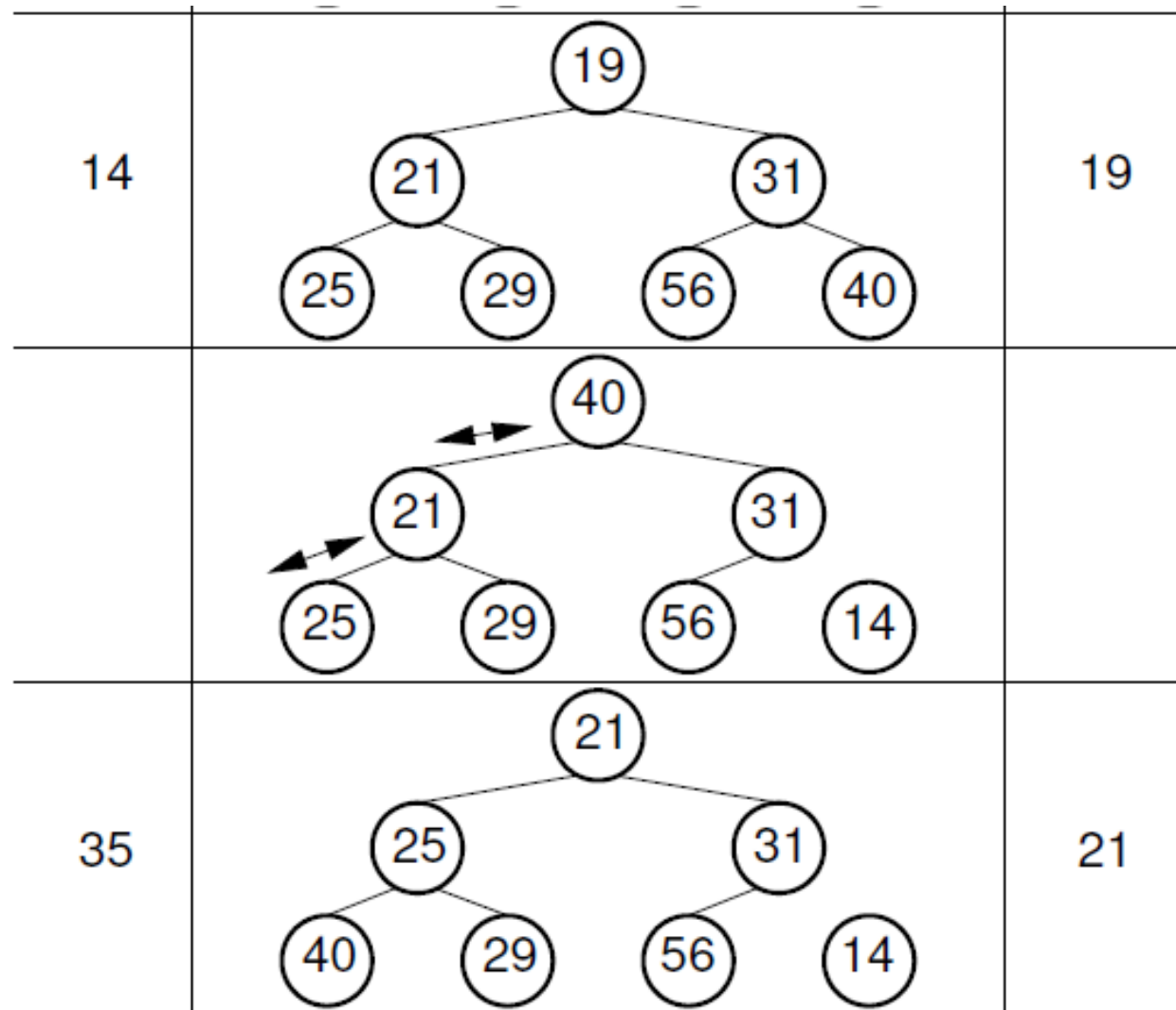
Multiway Merge sort

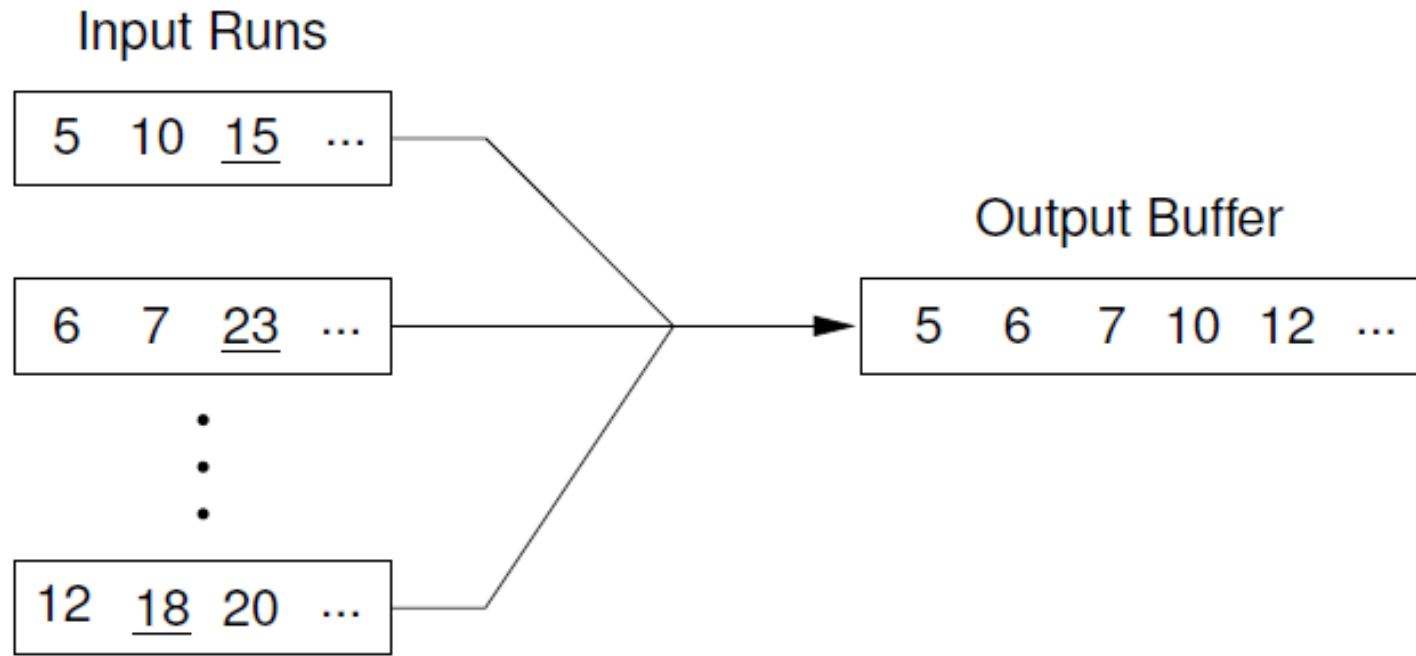
2/4



Multiway Merge sort

3/4



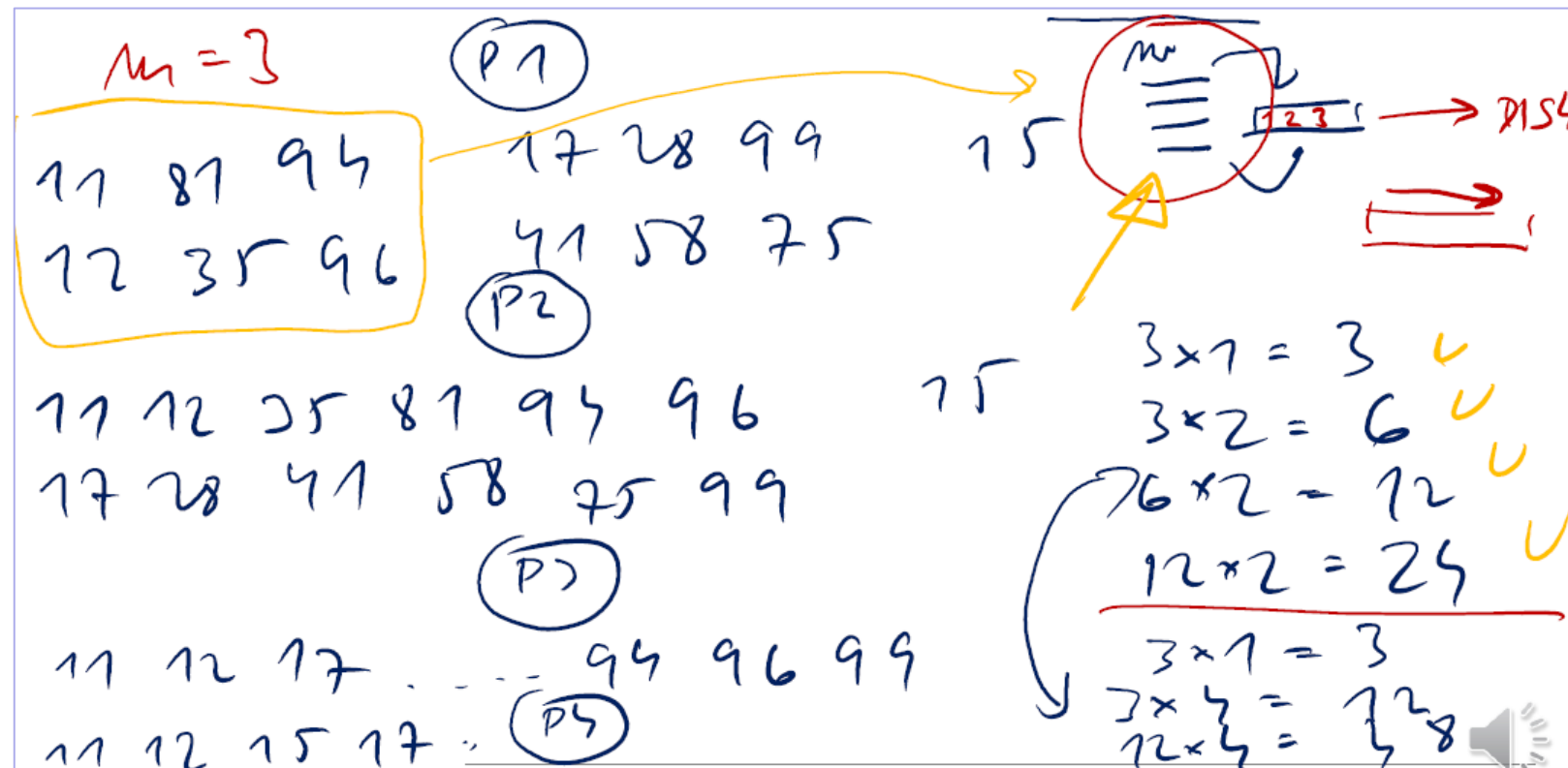


Příklad, externí Merge sort

Řešení – viz Kontrolní otázky, Řešené příklady

Seřadíte data: 81, 94, 11, 96, 12, 35, 17, 99, 28, 58, 41, 75, 15

dvoucestným **externím algoritmem Merge sort**. Jaký by byl rozdíl, kdybychom pro stejná data použili algoritmus čtyřcestný?



Příklad, řazení 1

```
void Sort(int array[], int size) {  
    for (int i = 0; i < size - 1; i++) {  
        int maxIndex = i;  
        for (int j = i + 1; j < size; j++) {  
            if(array[j]>array[maxIndex]) maxIndex = j;  
        }  
        int tmp = array[i];  
        array[i] = array[maxIndex];  
        array[maxIndex] = tmp;  
    }  
}
```

- A. Selection sort
- B. Insertion sort
- C. Bubble sort
- D. Merge sort
- E. Heap sort
- F. Quick sort
- G. Bucket sort

Příklad, řazení 2

```
void Sort(int array[], int size) {  
    for (int i = 0; i < size - 1; i++) {  
        int j = i + 1;  
        int tmp = array[j];  
        while (j > 0 && tmp > array[j-1]) {  
            array[j] = array[j-1];  
            j--;  
        }  
        array[j] = tmp;  
    }  
}
```

- A. Selection sort
- B. Insertion sort
- C. Bubble sort
- D. Merge sort
- E. Heap sort
- F. Quick sort
- G. Bucket sort

Příklad, řazení 3

```
void Sort(int * array, int size){  
    for(int i = 0; i < size - 1; i++){  
        for(int j = 0; j < size - i - 1; j++) {  
            if(array[j+1] < array[j]){  
                int tmp=array[j + 1];  
                array[j + 1]=array[j];  
                array[j] tmp;  
            }  
        }  
    }  
}
```

- A. Selection sort
- B. Insertion sort
- C. Bubble sort
- D. Merge sort
- E. Heap sort
- F. Quick sort
- G. Bucket sort

Příklad, řazení 4

```
void Sort(int array[],int aux[],int left,int right){  
    if (left == right) return;  
    int middleIndex = (left + right)/2;  
    Sort(array, aux, left, middleIndex);  
    Sort(array, aux, middleIndex + 1, right);  
    Zpracuj(array, aux, left, right);  
    for (int i = left; i <= right; i++) {  
        array[i] = aux[i];  
    }  
}
```

- A. Selection sort
- B. Insertion sort
- C. Bubble sort
- D. Merge sort
- E. Heap sort
- F. Quick sort
- G. Bucket sort

Příklad, řazení 5

```
void Sort(int array[], int size){  
    for (int i = size/2 - 1; i >= 0; i--) {  
        Restructure(array, size - 1, i);  
    }  
    for (int i = size - 1; i > 0; i--) {  
        swap(array, 0, i);  
        Restructure(array, i - 1, 0);  
    }  
}
```

- A. Selection sort
- B. Insertion sort
- C. Bubble sort
- D. Merge sort
- E. Heap sort
- F. Quick sort
- G. Bucket sort

Příklad, řazení 6

```
void Sort(int array[], int left, int right){  
    if(left < right){  
        int boundary = left;  
        for(int i = left + 1; i < right; i++){  
            if(array[i] > array[left]){  
                swap(array, i, ++boundary);  
            }  
        }  
        swap(array, left, boundary);  
        Sort(array, left, boundary);  
        Sort(array, boundary+1, right);  
    }  
}
```

- A. Selection sort
- B. Insertion sort
- C. Bubble sort
- D. Merge sort
- E. Heap sort
- F. Quick sort
- G. Bucket sort

Příklad, řazení 7

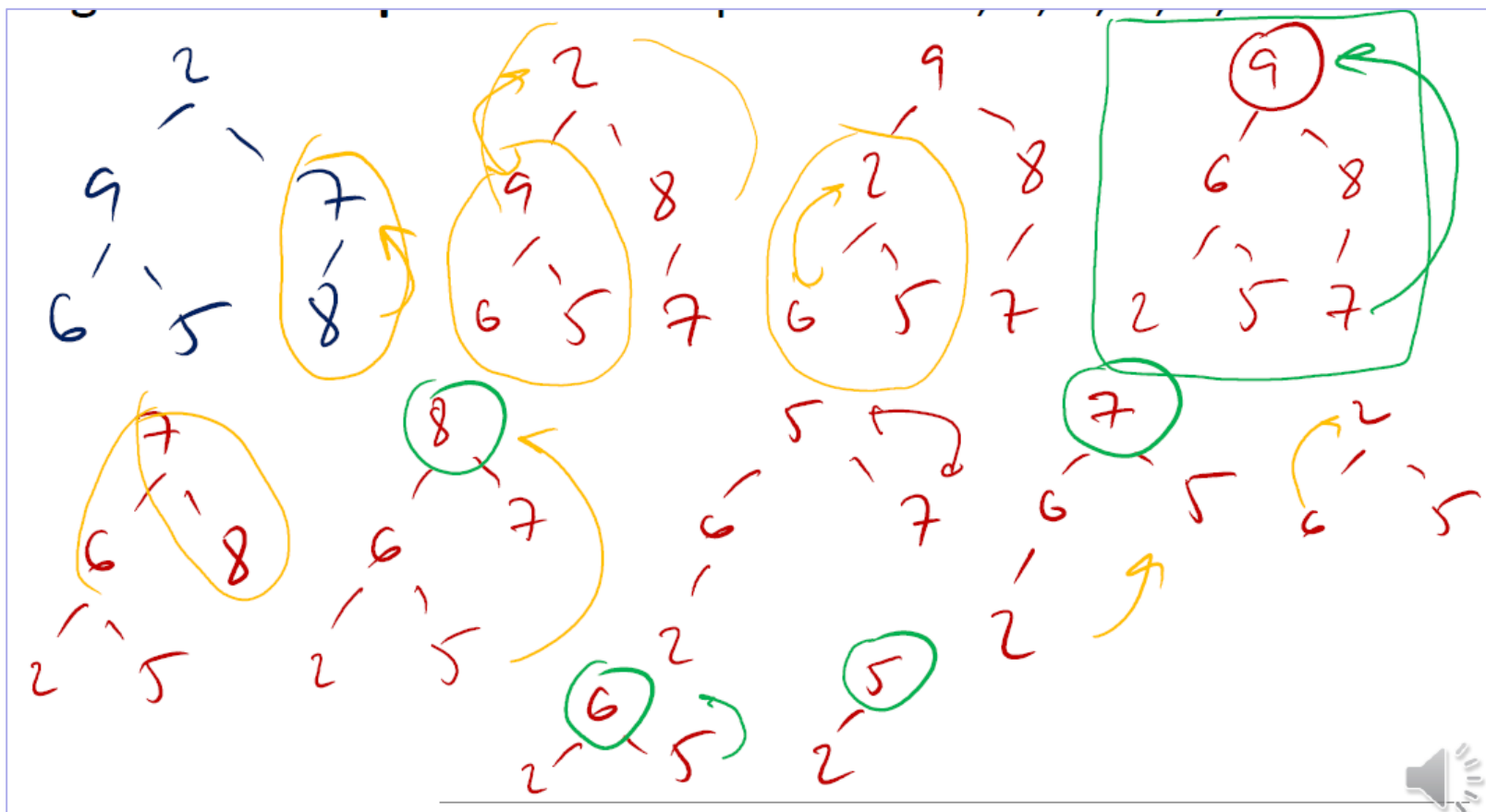
```
void Sort(int array[], int n)
{
    int i, j;
    int count[n];
    for (i = 0; i < n; i++)
        count[i] = 0;
    for (i = 0; i < n; i++)
        (count[array[i]])++;
    for (i = 0, j = 0; i < n; i++)
        for(; count[i]>0; (count[i])-- )
            array[j++] = i;
}
```

- A. Selection sort
- B. Insertion sort
- C. Bubble sort
- D. Merge sort
- E. Heap sort
- F. Quick sort
- G. Bucket sort

Příklad, řazení 8/1

Řešení – viz Kontrolní otázky, Řešené příklady

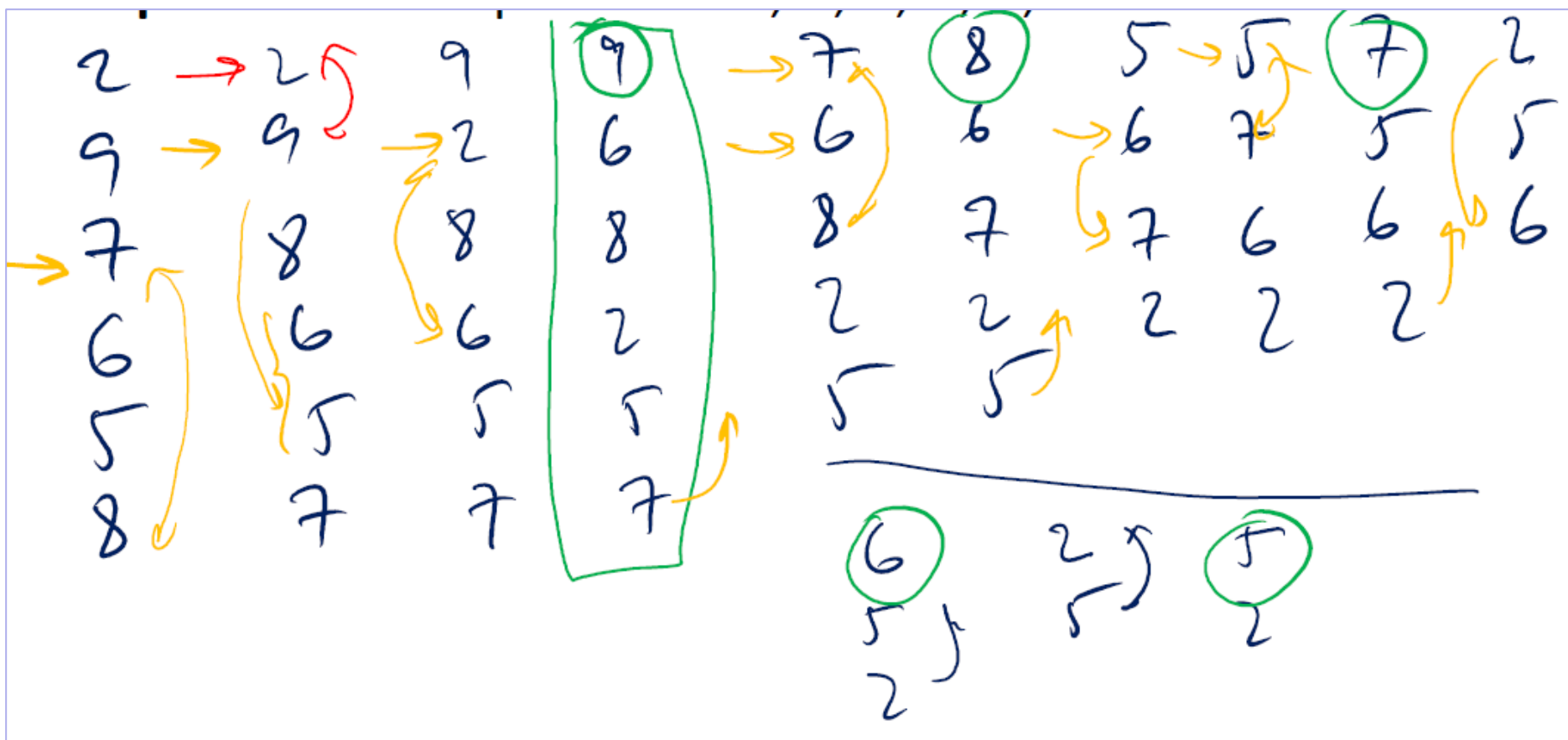
Pomocí **stromové reprezentace** vysvětlete aplikaci algoritmu **Heap sort** na vstupní data: 2, 9, 7, 6, 5, 8



Příklad, řazení 8/2

Řešení – viz Kontrolní otázky, Řešené příklady

Pomocí **reprezentace polem** vysvětlete aplikaci algoritmu **Heap sort** na vstupní data: 2, 9, 7, 6, 5, 8



Vyhledávání

- Vyhledávací problém
- Sekvenční vyhledávání
- Vyhledávání binárním půlením
- Binární vyhledávací stromy (BVC, binary search trees – BST)
 - základní vlastnosti
 - programová realizace
 - operace vyhledávání, nalezení minima a maxima
 - operace nalezení předchůdce a následníka
 - operace odstranění uzlu
 - operace vložení uzlu
- Adresní vyhledávání
 - přímý přístup
 - hašování